

面向专家混合 (MoE) 的无辅助损失负载均衡策略

Lean Wang^{1,2*}, Huazuo Gao¹, Chenggang Zhao¹, Xu Sun^{2,◊}, Damai Dai^{1,◊}

¹DeepSeek-AI²多媒体信息处理国家重点实验室, 北京大学计算机科学与技术学院
lean@pku.edu.cn, xusun@pku.edu.cn, damai.dai@deepseek.com

摘要

对于专家混合 (MoE) 模型, 专家负载不均会导致路由崩溃或增加计算开销。现有方法通常采用辅助损失来鼓励负载均衡, 但较大的辅助损失会在训练中引入不可忽视的干扰梯度, 从而损害模型性能。为在控制负载均衡的同时不在训练过程中产生不期望的梯度, 我们提出了 **Loss-Free Balancing**, 其特征是一种无需辅助损失的负载均衡策略。具体而言, 在进行 Top-k 路由决策之前, **Loss-Free Balancing** 会先对每个专家的路由得分施加逐专家的偏置。通过根据各专家的近期负载动态更新其偏置, **Loss-Free Balancing** 能持续保持专家负载的均衡分布。此外, 由于 **Loss-Free Balancing** 不会产生任何干扰梯度, 它也提升了通过 MoE 训练所能获得的模型性能上限。我们在规模高达 3B 参数的 MoE 模型上进行验证, 并在最多 200B Token 的训练数据上评估了 **Loss-Free Balancing** 的性能。实验结果表明, 与传统的由辅助损失控制的负载均衡策略相比, **Loss-Free Balancing** 在性能和负载均衡两方面都更优。

1 引言

专家混合 (MoE) 架构已成为在大型语言模型 (LLMs) 中扩展参数时管理计算成本的前景方案。在基于 Transformer 的模型 (Vaswani 等, 2017 年) 中的最新应用, 已成功将语言模型扩展到相当大的规模 (Shao 等, 2024 年; DeepSeek-AI 等, 2024 年; Dai 等, 2024 年; Fedus 等, 2021 年; Lepikhin 等, 2020 年), 从而带来显著的性能提升。然而, 训练 MoE 模型总是面临负载不均衡的情况, 这可能导致路由崩塌 (Shazeer 等, 2017 年) 或增加计算开销 (Fedus 等, 2021 年; Lepikhin 等, 2020 年; Shazeer 等, 2017 年)。为避免不均衡的路由, 现有方法 (Fedus 等, 2021 年; Lepikhin 等, 2020 年) 通常使用辅助损失以鼓励专家负载的均衡。尽管辅助损失可以在训练期间缓解负载不均衡, 但它也会引入与语言建模目标函数相冲突的非期望梯度。这些干扰梯度会损害模型性能, 因此现有 MoE 方法总是需要在负载均衡与模型性能之间进行权衡。

在本文中, 我们提出 **无损平衡**, 一种无辅助损失的负载均衡策略, 旨在在不引入干扰梯度的情况下保持对专家负载均衡的控制。无损平衡的特点是一个由令牌路由与偏置更新组成的迭代过程。如下所示

* 在 DeepSeek-AI 实习期间的贡献。◊ 通讯作者。

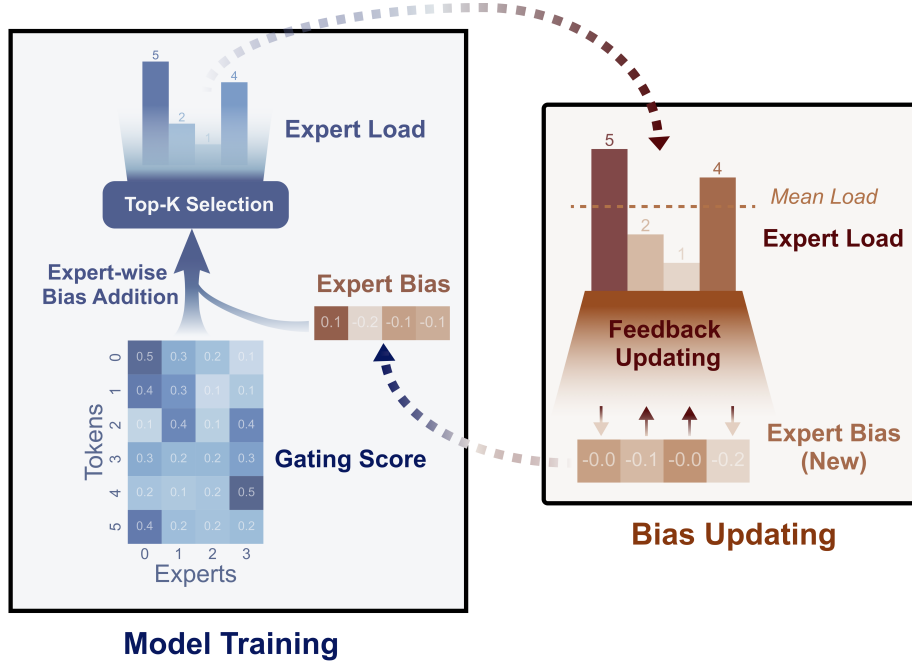


图1: Loss-Free Balancing 在每个训练步数中根据“带偏置的门控得分”选择专家，并在每个训练步数之后更新该按专家划分的偏置。

在图1中，在专家混合进行 Top-k 路由决策之前，无损平衡会先将逐专家偏置应用到原始路由得分上，生成带偏的门控得分，这些得分在训练过程中决定每个标记的实际路由目标。这些逐专家偏置将根据最近训练标记上观察到的专家负载持续更新：负载较重的专家其偏置会被降低，负载较轻的专家其偏置会被提高。通过这种动态更新策略，无损平衡确保带偏的门控得分能够持续导向均衡的路由结果。与由辅助损失控制的负载均衡策略相比，无损平衡不会引入干扰主要语言建模目标函数的不期望梯度，因此其训练过程噪声更少且更友好。

为了验证 Loss-Free Balancing 的性能，我们从零开始在 100B 个标记上训练 1B 参数的 MoE 语言模型，并在 200B 个标记上训练 3B 参数的 MoE 语言模型。实验结果表明，与传统的由辅助损失控制的模型相比，Loss-Free Balancing 产生了在验证损失上更优的 MoE 模型。同时，在保持性能优势的前提下，Loss-Free Balancing 还在全局和批次层面实现了显著更好的负载均衡，并且天然兼容专家并行，而专家并行通常用于训练极其大型的 MoE 模型。

2 背景

2.1 专家混合

当前主流的专家混合架构（Lepikhin 等，2020年；Fedus 等，2021年；Dai 等，2024年）将标准的 Transformer 模型中的 MLP 层替换为 MoE 层。在一个 MoE 层中，采用 Top-k 路由为每个标记选择专家。令 $\mathbf{u}_t$ 表示第 t -th 个标记输入到一个

N 个专家的 MoE 层, 输出 $\mathbf{h}_t$ 的计算如下:

$$\begin{aligned}\mathbf{h}_t &= \mathbf{u}_t + \sum_{i=1}^N g_{i,t} \text{FFN}_i(\mathbf{u}_t), \\ g_{i,t} &= \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise}, \end{cases} \\ s_{i,t} &= G(\mathbf{u}_t^T \mathbf{e}_i),\end{aligned}\tag{1}$$

其中 G 是一个非线性的门控函数, 且 $\mathbf{e}_i$ 是第 i 个专家的质心。

2.2 用于负载均衡的辅助损失

辅助损失 不受控的路由策略容易出现负载不均衡, 带来两个显著缺点。首先, 存在路由塌陷 (Shazeer 等, 2017 年) 的风险, 即模型持续只选择少量专家, 导致其他专家无法得到充分训练。其次, 当专家分布在多个设备上时, 负载不均衡会加剧计算瓶颈。为解决这些问题, 通常采用辅助损失 (Fedus 等, 2021 年; Lepikhin 等, 2020 年) 来控制负载均衡。对于长度为 T 的序列, 辅助损失定义为:

$$\begin{aligned}\mathcal{L}_{\text{Balance}} &= \alpha \sum_{i=1}^N f_i P_i, \\ f_i &= \frac{N}{KT} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ selects Expert } i), \\ P_i &= \frac{1}{T} \sum_{t=1}^T s_{i,t},\end{aligned}\tag{2}$$

其中 N 是专家总数, K 是为每个标记选择的专家数量, $s_{i,t}$ 是专家 i 对 Token t 的路由分数, f_i 表示已路由到专家 i 的标记所占比例, P_i 表示专家 i 的平均门控分数, α 是控制辅助损失强度的超参数。

负载均衡与模型性能之间的两难 上述的辅助损失可以促进负载均衡, 但作为额外的正则化项也会干扰语言建模训练。缺少辅助损失或较小的辅助损失系数 α 可能导致负载均衡较差, 而较大的 α 会损害训练, 从而使性能不佳。为说明这一两难, 我们在图2中给出了负载均衡与模型性能之间的关系。我们将 α 在 1e-2、1e-3、1e-4 和 0 之间取值, 并给出相应的 $\text{MaxVio}_{\text{global}}$, 其用于衡量负载均衡的程度, 计算细节见 § 4.1。如图所示, 较小的 α 会导致路由崩塌, 影响模型效率, 并可能导致部分专家学习或利用不足; 而较大的 α 虽能将负载均衡控制在合理范围内, 但会显著降低模型性能。为打破这一两难, 我们提出**无辅助损失平衡**作为解决方案, 其直接控制专家负载均衡, 但除语言建模损失产生的梯度之外, 不引入任何意料之外的梯度。

3 无辅助损失的负载均衡策略

为了得到一种更好的负载均衡替代方案, 且不直接干扰来自训练目标函数的主梯度, 我们提出**无辅助损失均衡**, 其根据各专家的负载均衡状况直接调整其门控得分。如图1所示, 我们将专家级偏置项 $\{\mathbf{b}_i\}_{i=1}^N$ 添加到每个专家的门控得分 $s_{i,t}$ 中, 并使用带偏置的得分来确定

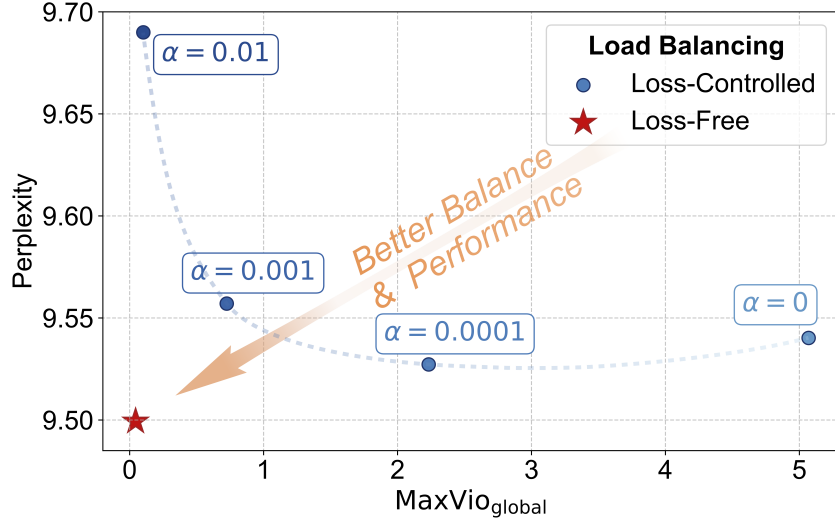


图2：在由辅助损失控制的训练中，负载均衡与模型性能之间的两难。较小的辅助损失系数 α 会导致负载均衡不佳，而较大的 α 会损害模型性能。相比之下，我们的 Loss-FreeBalancing 方法打破了这一两难局面。

算法 1: 在训练期间调整每个专家的偏置 b_i

输入: 专家混合模型 θ , 训练批次迭代器 B , 偏置更新率 u .

1. 为每个专家初始化 $b_i = 0$;
- for** 一个批次 $\{(\mathbf{x}_k, \mathbf{y}_k)\}_k$ 在 B **do**
 2. 在批次数据 $\{(\mathbf{x}_k, \mathbf{y}_k)\}_k$ 上训练专家混合模型 θ ，门控得分按式 (3) 计算;
 3. 统计每个专家被分配的标记 c_i 数量，以及平均数量 $\bar{c}_i$;
 4. 计算负载违反误差 $e_i = c_i - \bar{c}_i$;
 4. 将 b_i 更新为 $b_i = b_i + u * \text{sign}(e_i)$;

end

Output: 已训练的模型 θ , 对应的偏置 b_i

Top-k 选择:

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

请注意，专家偏置项 b_i 仅通过影响 Top-k 选择来用于调整路由策略。它不会被加到 $g_{i,t}$ 上，后者用于在计算专家混合层的最终输出时对所选专家的输出进行加权。

为得到合适的偏置，我们根据以下原则迭代地调整每个偏置 b_i ：当相应专家的负载较重时降低它，反之则提高。具体而言，对于每个 b_i ，我们持续监控其对应专家在上一批次上的负载。如果某个专家在上一批次负载较重，我们将降低其偏置；否则，我们将提高它。算法 1 描述了针对逐专家偏置的更新算法的细节。值得注意的是，我们基于历史的平衡状态来更新偏置，因为利用当前序列的负载信息会打破语言建模的因果约束，导致未来标记的信息泄漏。通过对偏置的动态调整，我们可以实现良好的专家负载均衡，而不会像辅助损失控制方法那样将噪声梯度直接引入模型。

表1: 不同负载均衡方法的比较。好的属性以绿色, 不良属性以红色。

负载均衡方法	均衡 专家负载	干扰 梯度	未来标记 泄漏
损失控制 (强辅助损失)	均衡	强	无泄漏
损失控制 (弱辅助损失)	失衡	weak	无泄漏
专家选择	均衡	none	存在泄漏
无辅助损失 (本文方法)	均衡	none	无泄漏

与其他负载均衡方法的比较。 为展示无辅助损失的负载均衡在理论上的优势, 我们将其与另外两种主流负载均衡方法进行比较, 即辅助损失控制方法 (Lepikhin 等, 2020年; Fedus 等, 2021年) 和 Expert Choice (EC) (Zhou 等, 2022年) 方法。正如 § 2.2 所述, 辅助损失控制方法在负载均衡与模型性能之间面临两难, 理想的权衡可能并不存在。对于 EC 方法, 它会打破语言建模的因果约束, 因为每个标记的目标专家取决于同一序列或批次中的未来标记。这将导致未来标记信息的泄漏, 从而破坏模型的泛化能力。表1 总结了不同负载均衡方法的特性。

4 实验

4.1 实验设置

模型架构。 我们采用 DeepSeekMoE (Dai 等, 2024年) 架构作为主干, 因为它显著优于诸如 GShard (Lepikhin 等, 2020) 等传统专家混合架构。与 GShard (Lepikhin 等, 2020) 相比, 它将专家划分得更细, 并将部分专家隔离为共享专家。与 DeepSeekMoE 略有不同, 在我们的主要实验中, 我们选择将 sigmoid 而非 Softmax 用作门控函数 G , 因为我们发现 sigmoid 基线优于 Softmax 基线。尽管如此, 我们仍在附录C中提供了关于 Softmax 门控的实验结果与讨论。我们的实验基于两种模型规模, 总参数为 1B 和 3B, 并且我们仅在 1B 缩放下调整偏置更新率。在 3B 缩放下的实验直接继承 1B 缩放下的最佳配置。受篇幅限制, 我们在附录A中提供了更多关于我们架构的细节。

训练设置 我们使用由 DeepSeek-AI 构建的多语言训练语料库, 来源广泛, 包括网页文本、数学材料、代码脚本以及已发表的文献。我们使用 HuggingFace Tokenizer¹ 来训练一种字节对编码 (BPE) (Sennrich 等, 2015年) 分词器, 词表大小为 32K。为得出可靠的结论, 我们在 100B 个标记上训练 1B 模型, 在 200B 个标记上训练 3B 模型, 以确保足够的训练量。我们分别为 1B 和 3B 模型采用余弦学习率调度器 (Loshchilov & Hutter, 2016年) 和多步学习率调度器 (Dai 等, 2024年)。受篇幅所限, 我们在附录 B) 中列出了关于训练设置和超参数的更多细节。

基线。 我们将无辅助损失负载均衡方法与传统的辅助损失控制方法进行比较。对于基线, 我们将辅助损失系数 α 设为 0.001, 以在模型性能与负载均衡之间取得合理的权衡 (见图 2)。由于 EC 方法存在未来标记泄漏问题, 我们未将其纳入对比, 并将在 § 5.2 中深入讨论。

¹<https://github.com/huggingface/tokenizers>

表2: 无损负载均衡在 1B 和 3B 模型上都实现了更低的困惑度和更好的负载均衡。使用验证集来计算这些指标 (详见附录B)。

模型规模	负载均衡方法	验证集困惑度	MaxVio _{global}
1B	损失控制	9.56	0.72
	无辅助损失	9.50	0.04
3B	损失控制	7.97	0.52
	无辅助损失	7.92	0.04

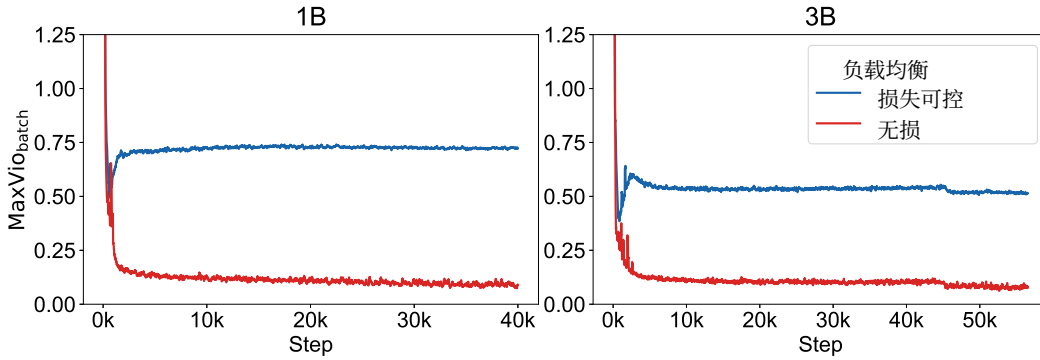


图3: 无辅助损失的负载均衡在大部分训练时间内保持了更好的负载均衡。这里, 为了可视化效果, 将 $\text{MaxVio}_{\text{batch}}$ 在相邻的 100 个步数上取平均。

指标。 我们从训练语料中留出一个验证集, 用于评估模型性能和负载均衡。对于模型性能, 我们采用困惑度作为指标。对于负载均衡, 我们引入一种称为“最大违例” (**MaxVio**) 的指标, 用于量化MoE层的负载均衡程度:

$$\text{MaxVio} = \frac{\max_i \text{Load}_i - \overline{\text{Load}_i}}{\overline{\text{Load}_i}}, \quad (4)$$

其中, Load_i 表示分配给第 i 位专家的令牌数量, 而 $\overline{\text{Load}_i}$ 表示在完美负载均衡下的期望专家负载。

MaxVio 有两种变体: **MaxVio_{global}** 和 **MaxVio_{batch}**。对于 **MaxVio_{global}**, 我们在整个验证集上统计 Load_i , 因此当批大小接近限制时, 它反映了专家利用的均衡程度以及效率上界。对于 **MaxVio_{batch}**, 我们在每个训练批次上统计 Load_i , 因此它与训练效率更相关。为简洁起见, 本文其余部分中, 我们报告跨所有层平均的 **MaxVio**, 作为整个模型的负载均衡度量。

4.2 主要结果

表2展示了使用辅助损失或我们的无辅助损失负载均衡策略训练的 1B 和 3B MoE 模型的验证困惑度和 **MaxVio_{global}**。如表中所示, 与由辅助损失控制的方法相比, 我们的 **Loss-Free Balancing** 在 1B 和 3B 模型上都取得了更好的困惑度和显著更好的全局负载均衡。此外, 为了呈现训练期间的负载均衡情况, 我们在图3中提供了随训练步数变化的 **MaxVio_{batch}** 的负载均衡曲线, 表明 **Loss-Free Balancing** 在负载均衡方面具有持续优势。总之, 我们的 **Loss-Free Balancing** 方法在训练期间避免对梯度造成干扰, 并有效控制负载均衡, 打破了 MoE 训练中负载均衡与模型性能之间的两难。

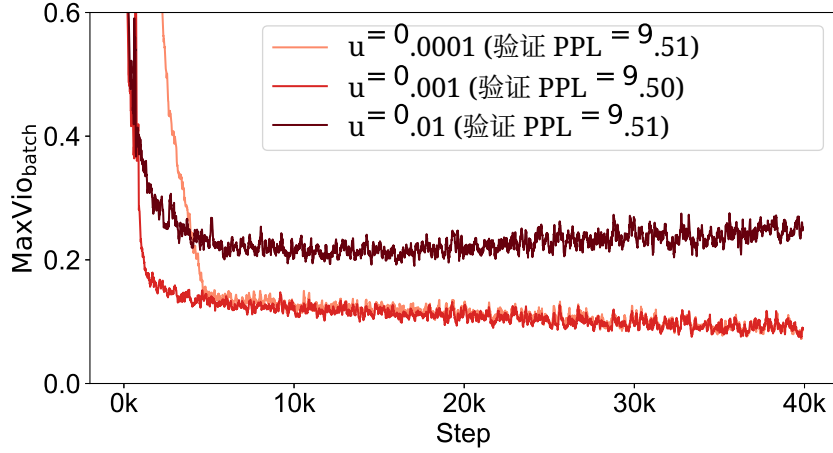


图4: 更新速率对训练负载均衡的影响。较低的更新速率在训练早期表现出较差的负载均衡, 而较高的更新速率在后期会恶化负载均衡。验证 PPL 表示验证困惑度。

表3: 该变体 $b_i = b_i + u * e_i$ 略微改善了负载均衡, 但在模型性能上未见提升。

方法	困惑度	MaxVio _{global}
$b_i = b_i + u * \text{符号函数}(e_i), u = 0.001$	9.50	0.044
$b_i = b_i + u * e_i, u = 0.01$	9.53	0.028
$b_i = b_i + u * e_i, u = 0.001$	9.51	0.036
$b_i = b_i + u * e_i, u = 0.0001$	9.51	0.040

4.3 BiasUpdate 算法的实证研究

我们对更新速率和偏置更新算法的变体进行了实证研究, 以验证我们在主要实验中采用的最优配置。

更新速率。 在算法 1 中, 更新速率 u 控制专家偏置 $\{b_i\}_{i=1}^N$ 收敛到“合适的偏置”的速度。图4 显示, 过低的更新速率 $u = 0.0001$ 可能导致收敛缓慢, 而不必要地过高的更新速率 $u = 0.01$ 会在训练后期引起专家偏置 b_i 的不良波动, 从而在该阶段恶化负载均衡。这两种情况都会损害性能。一个合适的选择是 $u = 0.001$, 其在训练的均衡性和验证困惑度方面表现良好。

更新规则。 我们研究了逐专家偏置的另一种更新规则。具体而言, 我们尝试将 $b_i = b_i + u * \text{sign}(e_i)$ 的更新规则改为 $b_i = b_i + u * e_i$, 以鼓励违反误差较高的专家的偏置变化得更快。尽管该变体略有改善负载均衡, 但并未带来更好的性能, 如表3所示。因此, 我们保留 sign 版本。

乘性偏置。 除了将逐专家的偏置加入门控分数之外, 使用乘性偏置也是一种可能的变体:

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} * b_i \in \text{Topk}(\{s_{j,t} * b_j \mid 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise}, \end{cases} \quad (5)$$

表4: 与加性偏置相比, 乘性偏置的负载均衡相近, 但性能略差。

方法	困惑度	MaxVio _{global}
加性偏置, $u = 0.001$	9.50	0.044
乘性偏置, $u = 0.01$	9.52	0.041
乘性偏置, $u = 0.001$	9.52	0.036
乘性偏置, $u = 0.0001$	9.54	0.048

这些 $\{b_i\}_{i=1}^N$ 可以通过与算法 1 类似的过程进行更新, 但应将它们初始化为 1 而非 0。表 4 显示, 与使用加性偏置相比, 使用乘性偏置的模型性能略差, 且对负载均衡没有显著改进。基于这些发现, 我们得出结论: 对于我们的方法, 加性偏置是更合适的选择。

5 讨论

5.1 无损均衡与专家并行兼容

极其大规模的 MoE 模型常在训练或推理中采用专家并行 (Lepikhin 等, 2020年), 将专家分布到不同的设备上以降低内存需求。在这种情况下, 单次计算步骤中的数据负载均衡对效率至关重要。由于专家并行, 每个计算步骤涉及 `micro_batch_size * ep_data_parallel_size` 个样本, 我们将其称为**计算批次**。其中, `micro_batch_size` 表示在单个设备上一次梯度累积步骤所处理的样本数。

无辅助损失负载均衡能够实现近乎最优的全局负载均衡, 并且随着计算批大小的增大, 每个计算步骤中的负载均衡会越来越接近全局负载均衡。在图5中, 我们使用 `MaxViocomputation-batch` 指标考察计算批级的负载均衡。结果表明, 随着计算批大小的增大, 我们的无辅助损失负载均衡的负载均衡性始终在不断提升, 但当计算批次很大时, 辅助损失控制方法的负载均衡性则大致维持在一个恒定水平。由于专家并行会将计算批大小显著扩大 `ep_data_parallel_size` 倍, 无辅助损失负载均衡天然适配大规模 MoE 训练, 且其在负载均衡方面的优势会随专家并行规模的增大而进一步增强。

5.2 负载均衡与未来标记泄漏

对于因果语言模型, 负载均衡方法必须遵循语言建模的因果约束, 以避免未来标记泄漏。尽管传统的辅助损失控制平衡以及我们的无辅助损失负载均衡遵守这一约束, **Expert Choice (EC)** (Zhou 等, 2022年) 却违反了它。EC 通过为每个专家分配完全相同数量的标记来确保完美的负载均衡。然而, 这种做法本质上会导致严重的未来标记泄漏问题。

在 EC 中, 未来标记可以影响先前标记的专家分配。图 6 展示了信息如何可通过这种影响在序列内轻松传播。从理论上讲, 稀疏比率为 R 的 MoE 层 (每个标记的平均激活专家数 K 除以专家总数 N) 的标记分配, 每个标记可泄露超过 $K \log_2 \frac{1-R}{R}$ 比特的信息 (证明见附录 D.1)。对于一个 9 层的、含 16 个专家且平均每个标记激活 2 个专家的专家混合模型, 这相当于 50 比特, 足以使每个标记确定其后继标记是哪一个。

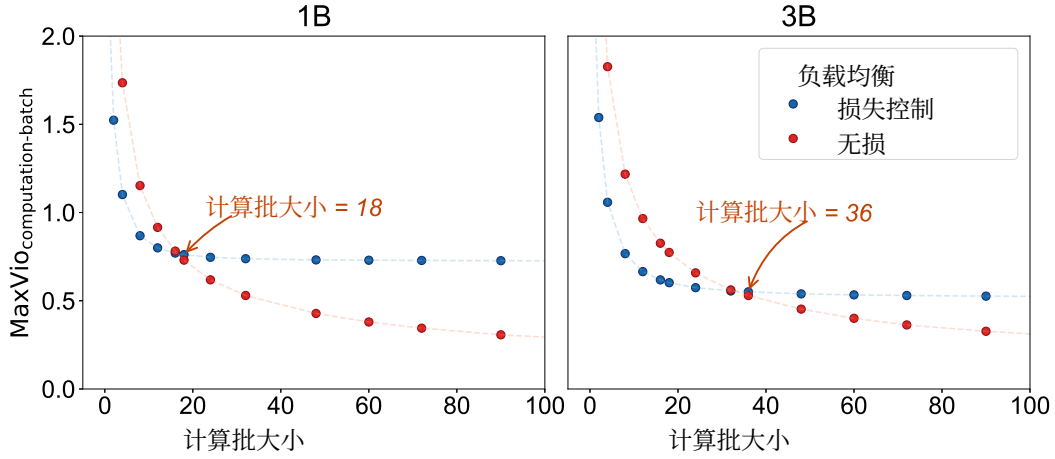


图5：随着计算批大小增加，无损均衡相比辅助损失训练实现了更好的均衡性，表明在使用中等规模的计算批次时更具优势。

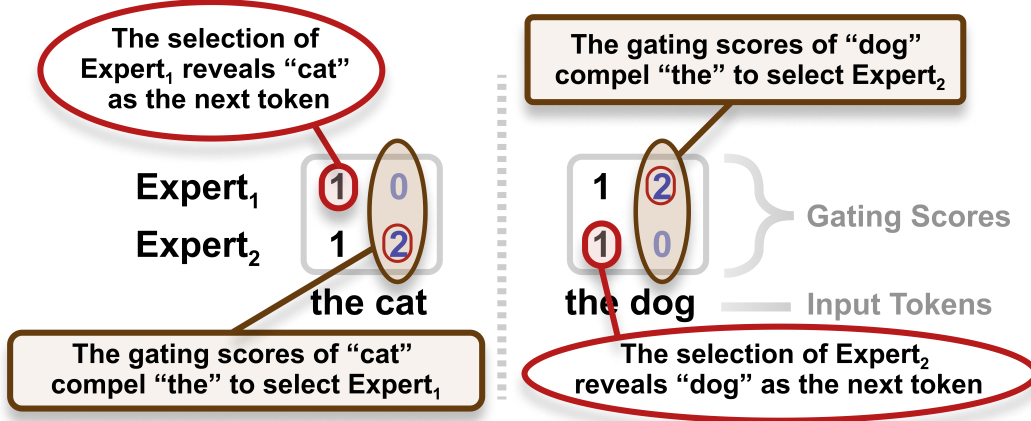


图 6：EC 中未来标记泄露的一个示例。未来标记会影响先前标记的专家分配。这样的分配可能帮助先前标记推断其后继标记的恒等函数。

我们设计了实验，以证明在真实的模型训练中存在未来标记泄露。(1) 我们将执行 Top-k 选择的块大小从 8192 个标记（4 个句子）减小到 512（1/4 个句子），以期暴露这种泄露。我们观察到异常的损失下降（约 10%），从而确认了泄露的存在。(2) 我们在 Top-k 选择步骤中跨块打乱标记，使泄露更难发生，并观察到该异常的损失下降得到缓解。关于 EC 信息泄露的详细实验结果见附录 D.2。

未来标记泄露是致命的，因为它破坏了模型的泛化能力，并阻碍对模型性能的可靠评估。因此，相比 EC，使用我们的无损均衡来扩展专家混合模型更为安全。

6 CONCLUSION

In this work, we introduced **Loss-Free Balancing**, a novel MoE load balance control method without introducing auxiliary-loss gradients. Loss-Free Balancing addresses the issue of traditional auxiliary-loss load balance control, which introduces additional gradients during training and potentially impairs model performance when enforcing load balance. Experiments conducted on 1B and 3B MoE models, trained on 100B and 300B tokens respectively, demonstrate that Loss-Free Balancing achieves better model performance and load balance compared to the traditional auxiliary-loss training.

REFERENCES

- Damai Dai, Chengqi Deng, Chenggang Zhao, Runxin Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *ArXiv*, abs/2401.06066, 2024. URL <https://api.semanticscholar.org/CorpusID:266933338>.
- DeepSeek-AI, Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y. Wu, Yukun Li, Huazuo Gao, Shirong Ma, Wangding Zeng, Xiao Bi, Zihui Gu, Hanwei Xu, Damai Dai, Kai Dong, Liyue Zhang, Yishi Piao, Zhibin Gou, Zhenda Xie, Zhewen Hao, Bing-Li Wang, Jun-Mei Song, Deli Chen, Xin Xie, Kang Guan, Yu mei You, Aixin Liu, Qiushi Du, Wenjun Gao, Xuan Lu, Qinyu Chen, Yaohui Wang, Chengqi Deng, Jiashi Li, Chenggang Zhao, Chong Ruan, Fuli Luo, and Wenfeng Liang. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *ArXiv*, abs/2406.11931, 2024. URL <https://api.semanticscholar.org/CorpusID:270562723>.
- William Fedus, Barret Zoph, and Noam M. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23:120:1–120:39, 2021. URL <https://api.semanticscholar.org/CorpusID:231573431>.
- Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam M. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *ArXiv*, abs/2006.16668, 2020. URL <https://api.semanticscholar.org/CorpusID:220265858>.
- Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv: Learning*, 2016. URL <https://api.semanticscholar.org/CorpusID:14337532>.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *ArXiv*, abs/1508.07909, 2015. URL <https://api.semanticscholar.org/CorpusID:1114678>.
- Zhihong Shao, Damai Dai, Daya Guo, Bo Liu, and Zihan Wang. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *ArXiv*, abs/2405.04434, 2024. URL <https://api.semanticscholar.org/CorpusID:269613809>.
- Noam M. Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ArXiv*, abs/1701.06538, 2017. URL <https://api.semanticscholar.org/CorpusID:12462234>.
- Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017. URL <https://api.semanticscholar.org/CorpusID:13756489>.

Table 5: Model architecture.

hyper-parameters	1B	3B
Vocab size	32064	32064
Hidden size	1024	1280
Attention heads	8	10
MoE layers	9	11
Granularity ($\frac{d_{ff}}{d_{expert}}$)	$\frac{16}{3}$	4
Shared experts	2	2
Routed experts	64	64
Activated routed experts	6	6

Yan-Quan Zhou, Tao Lei, Han-Chu Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. *ArXiv*, abs/2202.09368, 2022. URL <https://api.semanticscholar.org/CorpusID:247011948>.

A MODEL ARCHITECTURE

We employ the DeepSeekMoE (Dai et al., 2024) architecture as the backbone, which introduces shared experts to mitigate knowledge redundancy among routed experts:

$$\mathbf{h}_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t), \quad (6)$$

where r denotes the routed experts, while s the shared experts. DeepSeekMoE replaces all FFN layers with MoE layers, except the dense FFN layer just after the input embedding layer.

The detailed architecture hyper-parameters are listed in Table 5.

B TRAINING SETTINGS

Following the work of Dai et al. (2024), we initialize all learnable parameters with a standard deviation of 0.006, and set the maximum training sequence length to 2048.

For the 1B model, we employ a cosine learning rate scheduler with warmup, setting the learning rate to 1e-3, the minimum learning rate to 1e-4, and the warmup steps to 1000. The training batch size for the 1B model is set to 1152, resulting in a total of 40000 training steps (100B tokens).

For the 3B model, we use a multistep learning rate scheduler with stage steps = [45211, 50862, 56514] and corresponding stage learning rates of [7.8e-4, 2.47e-4, 7.8e-5]. The warmup steps for the 3B model are set to 2000. We use a training batch size of 1728 for the 3B model, resulting in a total of 56514 training steps (200B tokens).

For validation, we leave around 70M tokens from the training corpus as the validation set ($30 * 1B_batch_size * max_seq_len = 20 * 3B_batch_size * max_seq_len = 71M$ tokens).

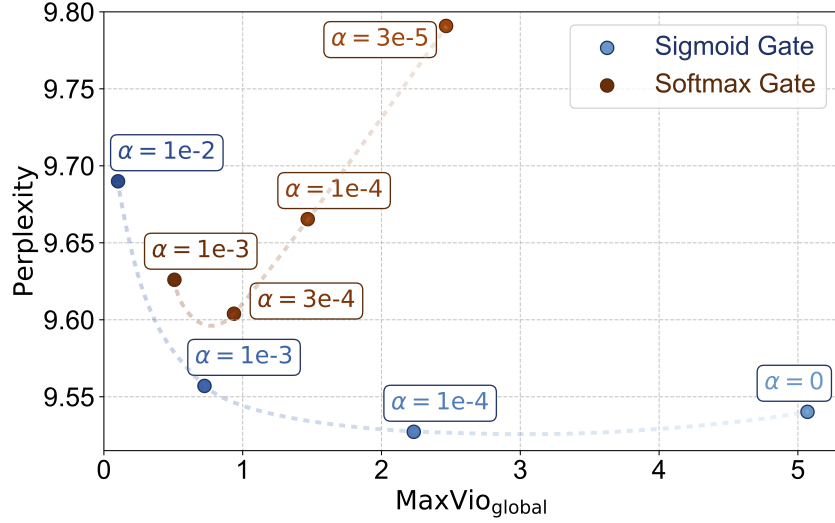


Figure 7: Comparison of the sigmoid gate baseline and the softmax gate baseline. The softmax gate exhibits higher perplexity under similar load balance conditions and is more sensitive to load imbalance compared to the sigmoid gate.

Table 6: For softmax gate, Loss-Free Balancing achieves a slightly lower perplexity while reaching a significantly better load balance compared to the auxiliary-loss training method.

Load Balancing	Perplexity	MaxVio _{global}
Loss-Controlled	9.604	0.937
Loss-Free	9.599	0.027

C EXPERIMENTS WITH SOFTMAX GATE

C.1 COMPARISON OF SIGMOID GATE BASELINE AND SOFTMAX GATE BASELINE

We compare the sigmoid gate baseline and the softmax gate baseline with varying auxiliary loss coefficients α on a 1B-sized model. As shown in Figure 7, the softmax gate exhibits higher perplexity under similar load balance conditions, and its performance is more sensitive to load imbalance compared to the sigmoid gate.

C.2 LOSS-FREE LOAD BALANCING WITH SOFTMAX GATE

Adjusting the per-expert bias for the softmax gate is more challenging due to the normalization property of softmax, which makes the score gap between two experts sensitive to the scores of other experts. In such a situation, we choose the $\mathbf{b}_i = \mathbf{b}_i + u * e_i$ variant to maintain load balance, where u is set to 1e-3. For the baseline, we choose $\alpha = 0.0003$, which yields the lowest perplexity for the softmax gate. The results are presented in Table 6, showing that Loss-Free Balancing achieves a slightly lower perplexity while maintaining significantly better load balance compared to the auxiliary-loss training method. Figure 8 confirms that Loss-Free Balancing maintains a superior load balance throughout most of the training process.

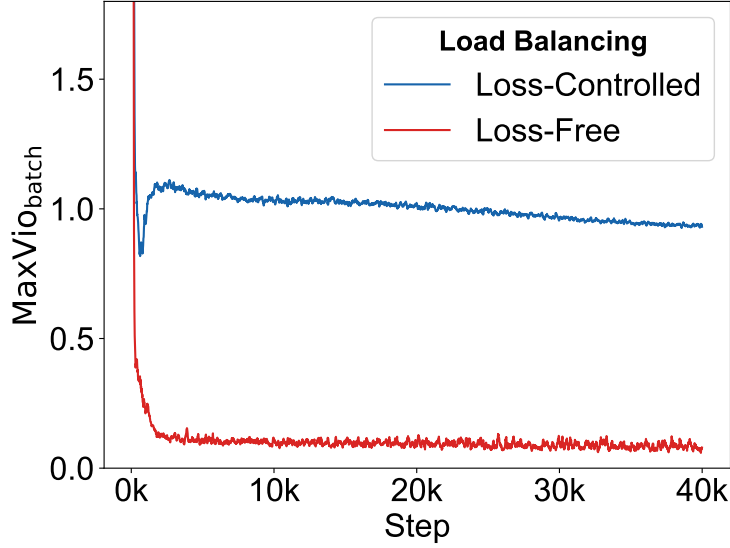


Figure 8: For softmax gate, Loss-Free Balancing maintains a superior load balance throughout most of the training process.

D FUTURE TOKEN LEAKAGE IN EXPERT CHOICE

D.1 PROOF FOR THEORETICAL LEAKAGE AMOUNT

Let $R = \frac{K}{N}$ denote the MoE sparsity. Here K denotes the average number of experts activated per token, and N is the total number of experts. For an MoE layer in Expert Choice, the maximum information leakage I (in bits per token), i.e., the information that the combinations of routing allocation can carry is:

$$\begin{aligned}
 I &= \log_2 \left(\frac{\left(\frac{KT}{N}\right)^N}{T} \right) \\
 &> N \log_2 \frac{\left(\left(1 - \frac{K}{N}\right)T\right)^{\frac{K}{N}T}}{\left(\frac{KT}{N}\right)^{\frac{K}{N}T}} \\
 &= K \log_2 \frac{1 - R}{R}.
 \end{aligned} \tag{7}$$

For a model with a sparse ratio $R = \frac{2}{16} = 0.125$ and 9 MoE layers, the total leakage information is more than 50 bits per token.

D.2 EXPERIMENTAL EVIDENCE

We investigate the potential future token leakage of the Expert Choice by varying the chunk size used for experts' top- k selection, ranging from 512 tokens to 8192 tokens.² We train a 2B MoE model on 100B tokens. The results, shown in Table 9, reveal two key findings:

1. Using a small chunk size of 512 leads to an abnormal loss drop, which can be attributed to significant future token leakage. A smaller chunk size allows the model to more easily exploit information from future tokens within the chunk during training.

²A chunk size of 2048 tokens means performing top- k selection inside a sentence, while 512 tokens correspond to a quarter of a sentence and 8192 tokens to four sentences.

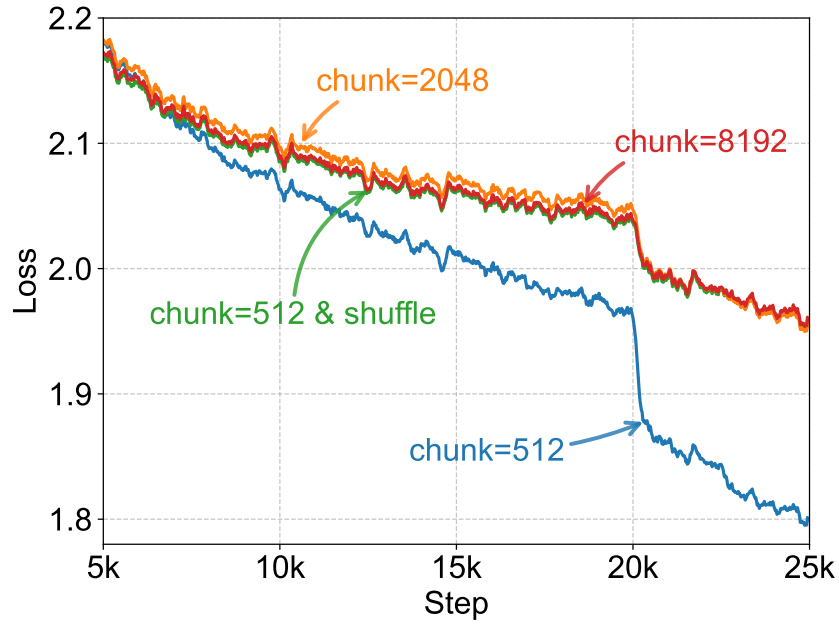


Figure 9: Comparison of Expert Choice with different chunk sizes and shuffling. Expert Choice with a chunk size of 512 exhibits a significant loss drop compared to chunk sizes of 8192 or 2048. Shuffling tokens eliminates this loss drop, indicating the presence of future token leakage.

2. Shuffling tokens within a batch before chunking and selecting mitigates the observed loss drop. Such shuffling makes it more challenging for the model to utilize information leakage, as the future tokens are no longer in their original context. This finding supports the hypothesis that the loss drop originates from the model's accessing and exploiting future token information.